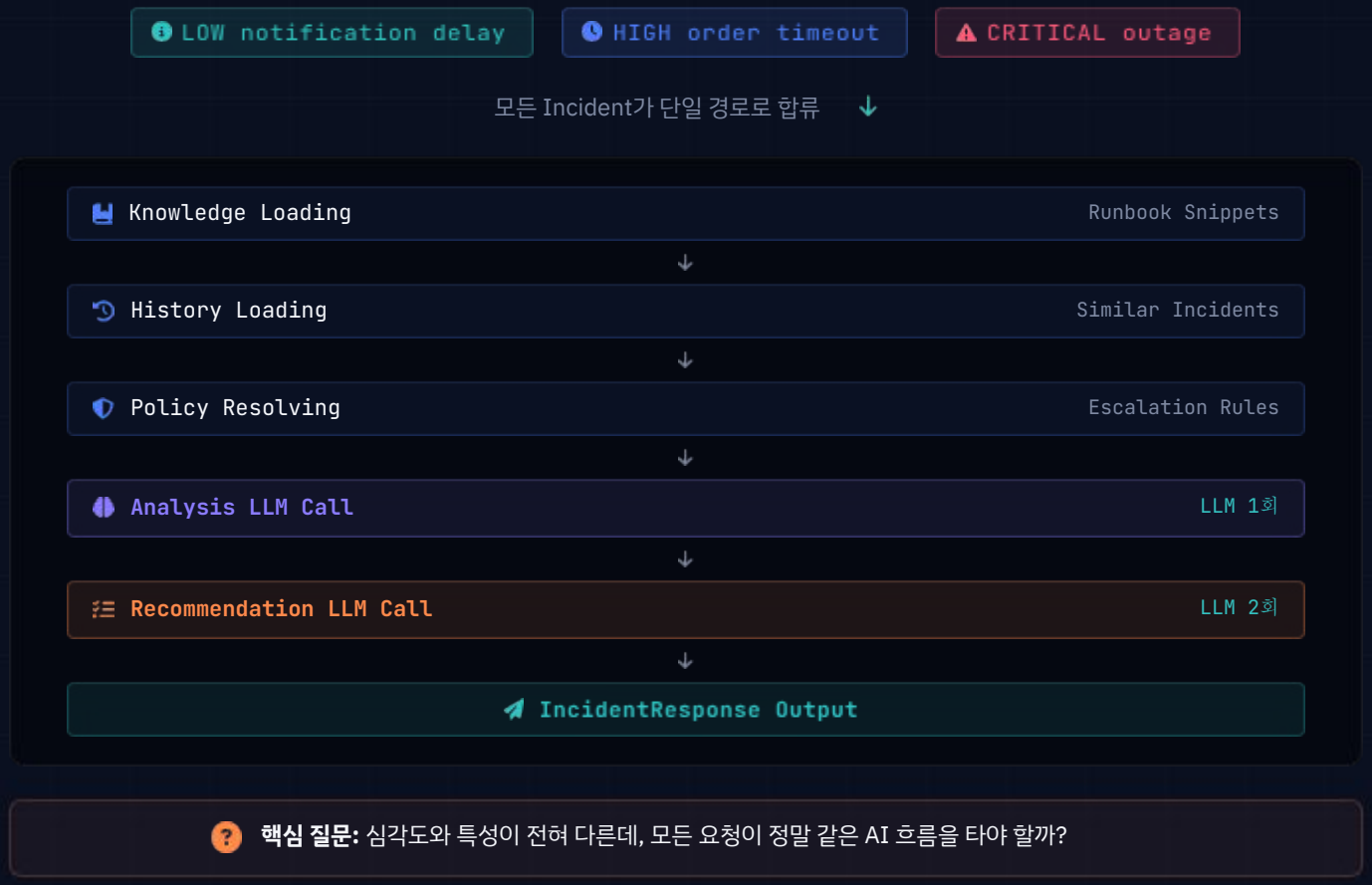
SECTION 3 | LECTURE 8

수업 9. Routing Workflow로 Incident 흐름 통제하기

요청 특성에 따라 실제 실행 경로 선택하기

통제 전 한계: 지난 수업에서 확인한 선형 흐름



핵심 질문: 심각도와 특성이 전혀 다른데, 모든 요청이 정말 같은 AI 흐름을 타야 할까?

흐름 통제의 실체: Routing을 적용하면 실제 경로가 달라진다

ROUTING INPUTS severity · sourceSystem · symptoms (구조화된 3대 평가 입력)

3-Way Branch Decision

ROUTE 01 LLM 0회

QUICK_TRIAGE

LOW + notification-service + 모든 symptom이 허용된 delay 계열일 때 선택

- **결정론적 응답:** 모델 호출 없이 즉시 생성
- **지연/비용 제로:** 대기 시간 및 토큰 소모 배제
- **보수적 경계:** 허용 외 증상 포함 시 제외

Fast Deterministic Triage

ROUTE 02 LLM 2회

STANDARD_ANALYSIS

명확한 QUICK이나 CRITICAL 조건에 해당하지 않는 일반적인 Incident

- **문맥 조회:** knowledge + history + policy
- **분석 LLM:** 1차 인과 분석 generation
- **추천 LLM:** 2차 액션 추천 generation

Full Cognitive AI Pipeline

ROUTE 03 정책검증 + LLM 2회

CRITICAL_ESCALATION

CRITICAL severity 또는 payment failure, data loss 같은 중요 신호 우선 포착

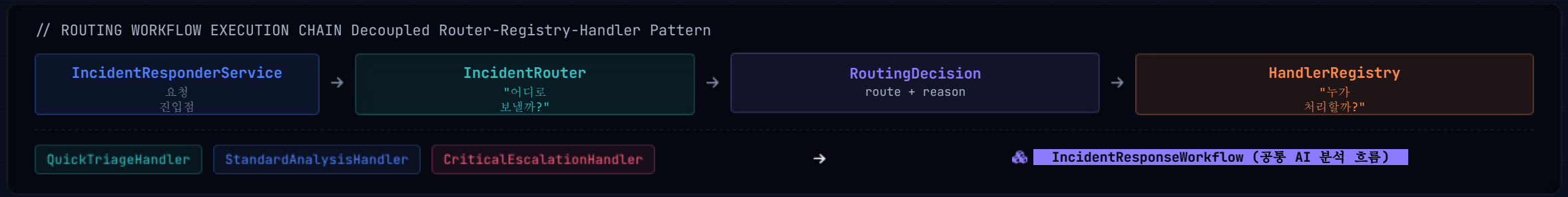
- **필수 policy 검증:** escalation policy 사전 확인
- **정책 미존재 시:** AI 분석 중단 (Fail-fast)
- **정책 존재 시:** 심화 AI 분석 및 추천 진행

Pre-Policy Guard + Escalation

Routing의 본질: route 이름을 부여하는 태깅이 아니라, 요청 특성에 따라 **실제 실행 경로를 바꾸는 흐름 통제**입니다.

* QUICK 좁게 허용 · CRITICAL 우선 판단

아키텍처 책임 분리: 코드에서는 누가 결정하고, 누가 실행할까?



ROUTER

어디로 보낼까?

요청의 속성(severity, system, symptoms)을 평가하여 최적 route 결정

DECISION

결과와 판단 근거

선택된 route뿐 아니라 디버깅을 위해 선택 이유(reason)를 객체로 기록

REGISTRY & HANDLER

누가 무엇을 실행할까?

Registry가 적합한 Handler를 조회하고, Handler가 해당 경로의 로직 실행

WORKFLOW

공통 AI 분석 파이프라인

STANDARD와 CRITICAL이 공유하는 knowledge, history, LLM 호출 흐름 캡슐화

설계 의사결정: 왜 단순한 Rule-based Routing인가?

CURRENT SELECTION 최적 선택 ✓

</> Rule-based Routing

조건이 명확한 결정론적 자바 규칙 기반 분기

- **결정론적 일관성:** 동일 입력에 항상 100% 동일 경로 보장
- **비용·지연 제로:** 추가 토큰 소비 및 모델 대기 시간 전무
- **직관적 테스트:** 단위 테스트 및 분기 이유 설명이 극도로 용이

명확한 조건 + 빠른 실행 + 투명한 제어

ALTERNATIVE 01 모호한 문맥용

LLM-based Routing

프롬프트 기반으로 LLM이 자연어 의도를 분류

- **의미 해석 장점:** 자연어 의미 해석이 필수적일 때 탁월
- **추가 비용·지연:** 분기를 위한 별도 LLM 호출 비용 발생
- **확률적 위험:** 오분류 가능성 및 Structured Output 검증 필요

자연어 의미 해석이 반드시 필요한 모호한 요청에 적합

ALTERNATIVE 02 대규모 정책용

Rule Engine (Drools 등)

전용 Rete 알고리즘 기반 복합 규칙 관리 엔진

- **정책 격리 관리:** 수백~수천 개 정책 및 동적 변경에 적합
- **과도한 복잡도:** 현재 규칙 수 대비 구조와 러닝커브 과다
- **운영 오버헤드:** 별도 DSL 및 런타임 엔진 유지보수 부담

정책 수가 방대하고 잦은 외부 변경이 필요할 때 검토

> **통제 설계 제1원칙:** LLM을 배제하는 것이 목적이 아니라, **결정론적으로 판단할 수 있는 문제를 굳이 확률적인 판단으로 바꾸지 않는다.**

*운영 환경: category·alert code 등 구조화 입력 권장

실습 검증 시나리오: 오늘 실습에서 확인할 것

PHASE 01. IDE

코드 전체 구조 확인

Router → Decision → Registry → Handler

PHASE 02. POSTMAN

Route & Reason 확인

QUICK, STANDARD, CRITICAL 응답 필드

PHASE 03. LANGFUSE

실제 실행 Trace 비교

LLM 호출 유무 및 Handler span 분기 검증

// EXPECTED VERIFICATION OUTCOMES Observable Path Differentiation

QUICK_TRIAGE	route-quick-triage 실행	[LLM generation 0건 (완전 배제)]
STANDARD_ANALYSIS	route-standard-analysis 실행	[Analysis + Recommendation LLM 2건]
CRITICAL_ESCALATION	required policy 검증 후 실행	[사전 정책 검증 통과 후 LLM 2건]

⚠ **QUICK 경계 테스트:** delay 증상이 포함되어도 허용되지 않은 다른 symptom이 섞이면 **QUICK이 아닌 STANDARD로 안전 분기**됨을 확인합니다.

* IDE 실습 전환 준비

🏁 오늘 정리 & 다음 수업 안내

TODAY SUMMARY

Routing Workflow로 흐름 통제 완결

- **선형 흐름 탈피:** 모든 Incident를 동일한 비용의 AI 경로로 보내지 않음
- **3대 경로 분기:** QUICK(결정론), STANDARD(일반), CRITICAL(정책검증)
- **결정론적 규칙:** severity, sourceSystem, symptoms 기준 투명하고 빠른 제어
- **실행 차이 검증:** Postman의 route 필드 및 Langfuse trace tree로 입증

"관찰 → 문제 발견 → 흐름 통제 → 재관찰"

UP NEXT LECTURE

다음 세션 미리보기 (수업 10)

경로는 나누었지만, STANDARD/CRITICAL 안에서 LLM에 얼마나 많은 컨텍스트를 보낼지는 아직 통제되지 않았습니다.

Flow Control: 어느 경로로 보낼 것인가? (오늘 완성)

Cost Control: 그 경로에서 얼마나 많은 정보를 허용할 것인가? (수업 10)

LECTURE 10 TOPIC

Context Budget으로 프롬프트 비용 통제하기 (Runbook/과거이력 캡핑)

Move to Cost Control ›